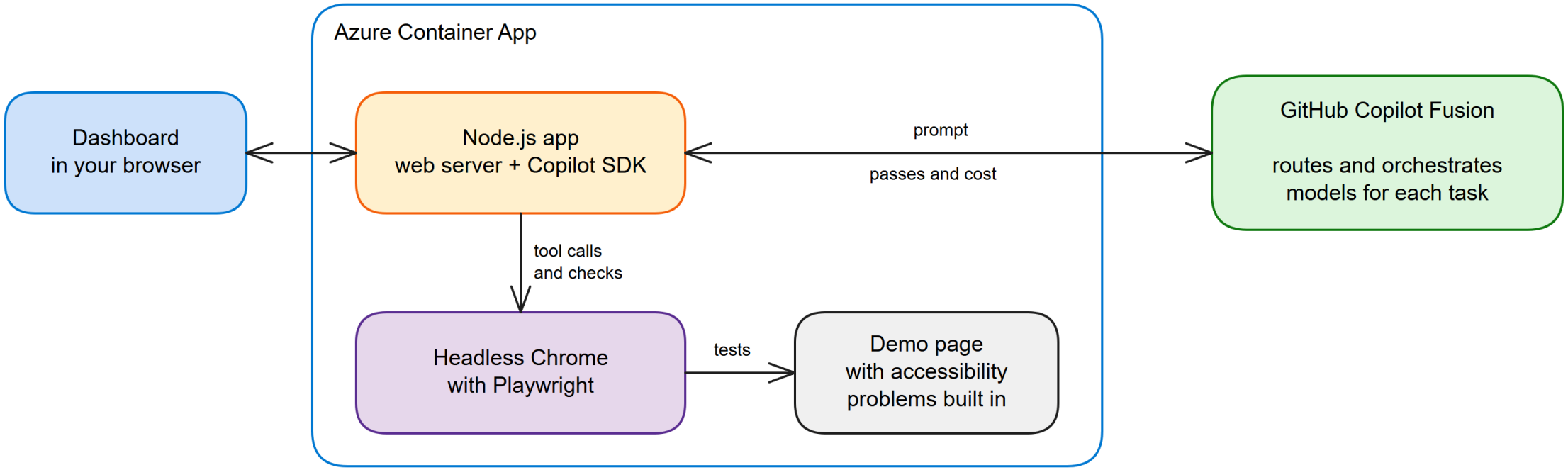
GitHub Copilot Fusion

Accessibility testing

Route models, inspect passes, and compare measured results and costs.



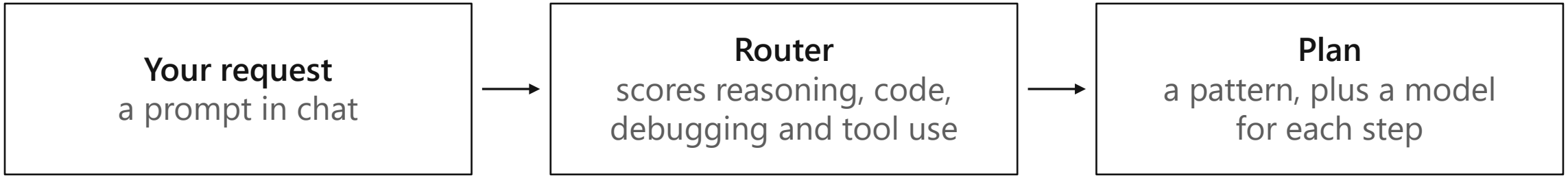
Generally available | Standard, High and Max effort | October 2026

Agenda

	Section	Slides
01	What Copilot Fusion is	3–4
02	Three patterns	5–8
03	How cost is counted	9–11
04	Testing a page for accessibility	12–16
05	What it cost, and try it yourself	17–19

What Copilot Fusion is

Routes and orchestrates models for each task at your chosen effort level.



- Generally available; select Copilot Fusion in the model picker
- Three effort levels: Standard, High and Max
- Each step can use a different model and reasoning effort
- If routing fails, it carries on with a fallback model

Choose Copilot Fusion and an effort level

Update your client, then select Copilot Fusion in the model picker.

Fusion Standard

Versatile models for everyday coding.

Fusion High

Capable models for challenging tasks and deeper reasoning.

Fusion Max

The most capable models for demanding tasks and complex reasoning.

Available on all Copilot plans in Visual Studio Code, the GitHub Copilot app, Copilot CLI and JetBrains IDEs. Your organization and enterprise model policies apply.

Billing: each selected model's standard rate. The launch announced a limited-time 30% model-cost discount for paid plans. The workshop displays SDK-reported charges; historical recordings are not repriced.

Launch announcement: October 28, 2026 (github.blog)

SINGLE



Simple isolated fixes.

- Syntax changes
- Trivial tag or attribute fixes

CASCADE



Tiered validation.

- WCAG linting
- Escalating checks

CRITIQUE

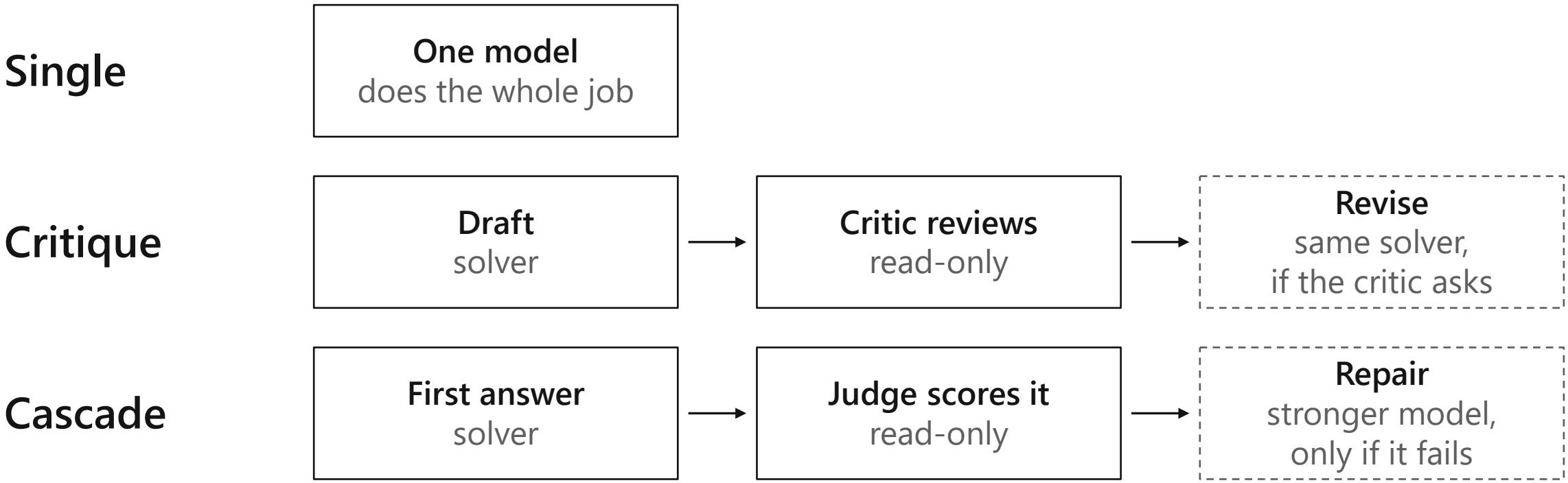


Deep independent review.

- Complex ARIA trees
- Compliance review

Three patterns

The router picks one of these for each request. You can't pick it directly.



Dashed steps only run when needed.

Critique or Cascade

Both add a second opinion. They use it differently.

	Critique	Cascade
Who reviews	A critic	A judge
What the review returns	Notes on the draft	A score: pass or fail
If the answer falls short	The same model revises it	A stronger model repairs it

A Critique fix stays with the same model. A Cascade fix brings in a stronger one, so it can cost more.

Behind the scenes

Reviews

- Run in a separate, read-only history
- Start from the answer and a summary
- Can't change your files
- Can miss checks that were never run

In Cascade

- A rejected answer is held back
- You only see the repaired answer
- File edits from the first try stay
- The repair can reuse or redo them

Reviews never enter your chat history, so they don't add to the size of later requests.

Two ways cost is counted

Premium requests (Legacy)

- Counted per prompt, not per token
- Each model has a multiplier
- Some models count 1, others 15
- Extra steps can add to it

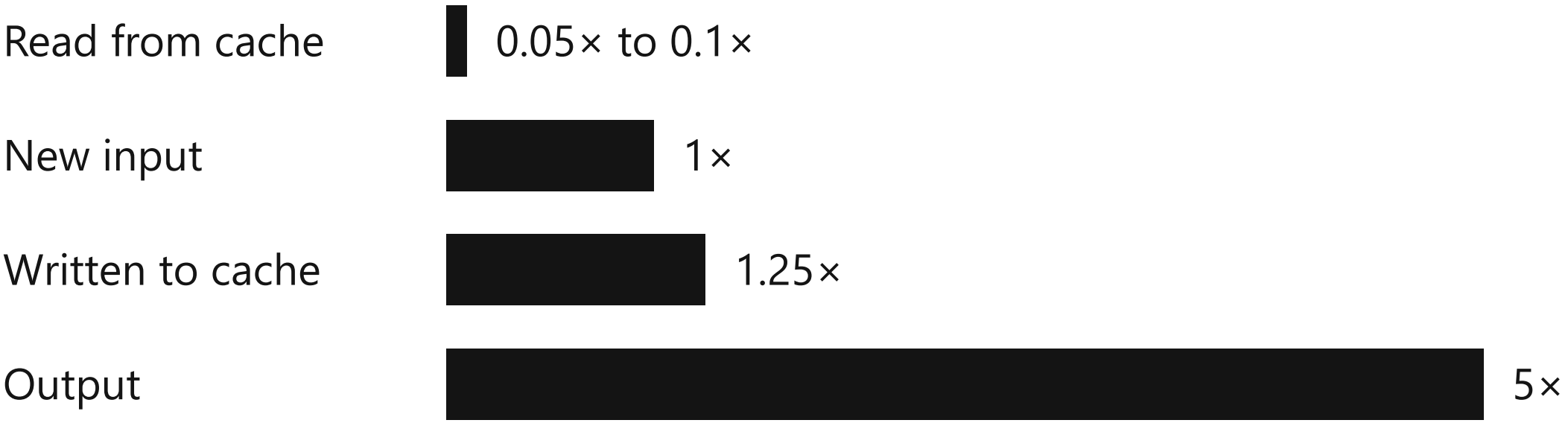
AIU (AI Units - June 1, 2026)

- Counts every token, in and out
- Each model has its own prices
- Logs record it as nano-AIU
- A billion nano-AIU make one AIU

The two don't always agree. A model can count as 1 premium request and still cost a lot per token.

How tokens are priced

Price per token, compared with new input, for the same model



Base prices also vary between models. The gap can be 100 times per token.

Where it saves, where it costs more

Saves

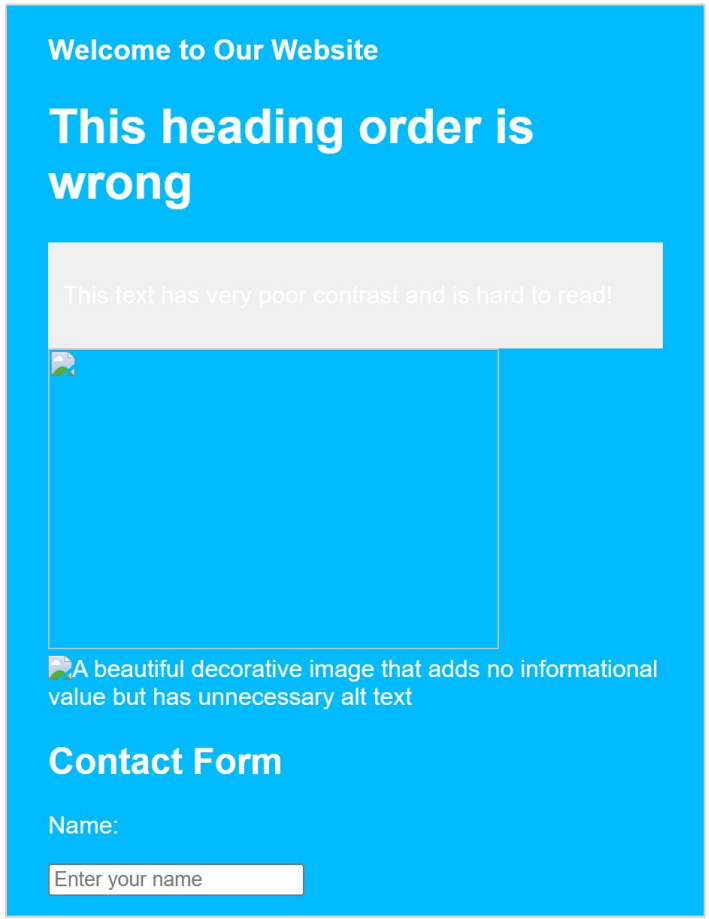
- Easy requests can go to a cheap model
- Reviews skip the full chat history
- Extra steps only run when needed
- Repeated context is read from cache

Costs more

- You pay for a rejected answer too
- It may pick an expensive model
- Long chats cost more per request

Your chat history goes with every request, so start a new chat for each test.

Testing a page for accessibility



A demo page with problems built in

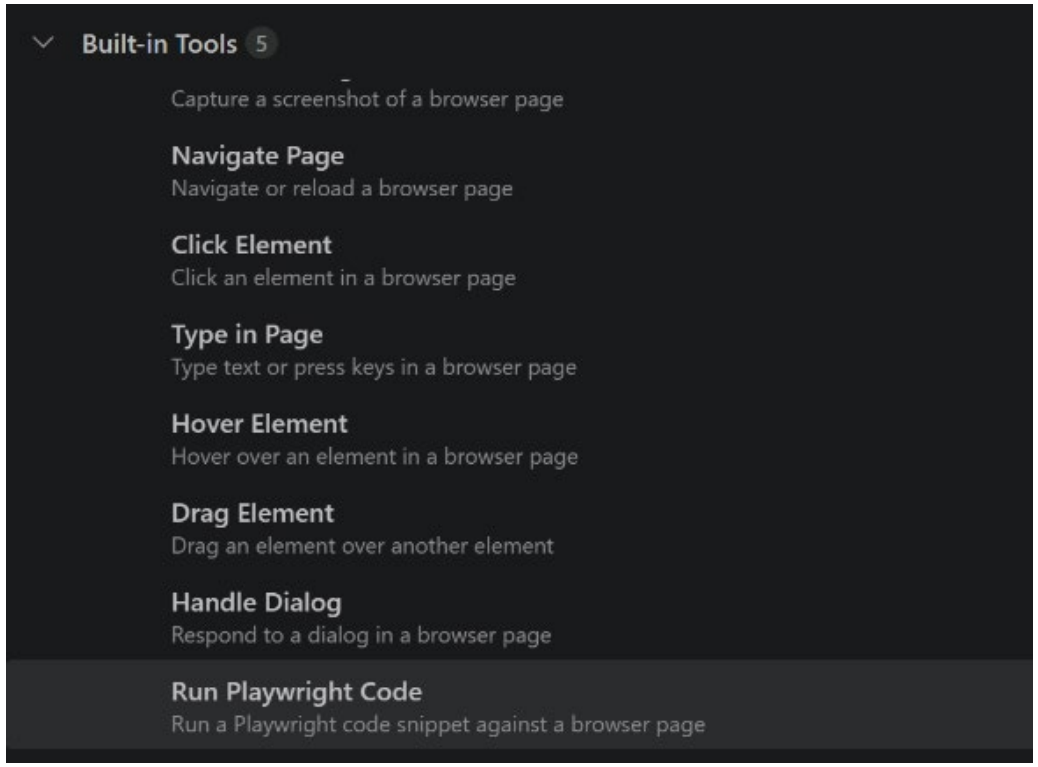
1. Open the page in VS Code's browser
2. Share it with chat
3. Pick Copilot Fusion and choose High effort
4. Send a prompt like this one

```
quickly test this page with the browser tools
```

It reads the page, runs Playwright checks and can press keys to test the keyboard. No terminal needed.

The browser tools

Built into VS Code. Find them in Agent Customizations, under Tools.



Open Browser Page
opens a URL, or a tab you share

Read Page
lists each element by role and name

Screenshot Page
shows what the page looks like

Navigate, Click, Type, Hover, Drag
use the page like a person would, keyboard included

Handle Dialog
answers alerts and prompts

Run Playwright Code
runs a short script for anything else

Run Playwright Code

Runs a short Playwright script on the page. The agent writes it, and you approve it.

A script can

- Read the DOM and computed styles
- Press keys and see where focus goes
- Resize it or turn on reduced motion
- Load axe-core into the page
- Return the values it measured

```
const stops = [];
for (let i = 0; i < 6; i++) {
  await page.keyboard.press('Tab');
  stops.push(await page.evaluate(
    () => document.activeElement.tagName));
}
return stops;
```

On the demo page it returned:

`A, INPUT, INPUT, INPUT, TEXTAREA, DIV`

The last stop is a div acting as the Submit button.

What a snippet can check

Each row is one snippet, run on the demo page in VS Code's browser.

Check	How	On the demo page
Contrast	Works out the ratio from colors	1.14:1, white on light gray
Language, landmarks	Looks for lang and main	Both are missing
Heading order	Lists the headings in order	H3, then H1, then H2
Keyboard	Presses Tab, notes each stop	Three stops are divs, not buttons
Reflow	Resizes to 320 px wide	Scrolls sideways, 348 px wide
Motion	Turns on reduced motion	The blinking text keeps going
Names and roles	Reads the accessibility tree	An image and a field with no name
axe-core	Loads axe and runs it	Flags contrast, lang, alt, labels

Use it as a first pass

It can check

- Contrast ratios
- Missing alt text and labels
- Heading order and landmarks
- Page language
- Basic keyboard use

Then

- Run axe-core
- Run Accessibility Insights
- Keep the CI checks on pull requests
- Keyboard and screen reader testing
- Check that alt text makes sense

Ask for measured values and screenshots, so you can check its work.

What the same test cost

The demo page and the same prompt, with one run of each pattern.

Pattern	Step	Model	AIU per 1M tokens	Premium ×	AIU used
Critique, Max	Draft	GPT-5.6 Luna	20 in, 120 out	1	0.94
	Critic	GPT-5.6 Terra	200 in, 1,200 out	1	0.28
Cascade, High	First answer	GPT-6.1 Sol	200 in, 1,000 out	1	141
	Judge	GPT-6 Luna	10 in, 50 out	1	0.2
	Repair	Claude Opus 5.5	400 in, 2,000 out	15	143
Single, Max	Whole job	GPT-6 Astra	1,000 in, 5,000 out	1	74

1.2 AIU
Critique, 2 premium requests

285 AIU
Cascade, 21 premium requests

74 AIU
Single, 1 premium request

Historical pre-GA runs; Default is now Fusion High. Models and rates can change.

Try it yourself

Open a new chat for each test, choose an effort level, then send the prompt.

Single

Fusion High effort

```
In one sentence, what does WCAG 1.1.1 require? No tools.
```

Critique

Fusion Max effort

```
Audit this page for WCAG 2.2 AA. Don't change any files.
List each problem and the rule it breaks.
```

Cascade

Fusion High effort

```
quickly test this page with the browser tools
```

Each prompt tends to get the pattern next to it, but the router decides.

How to tell which one you got

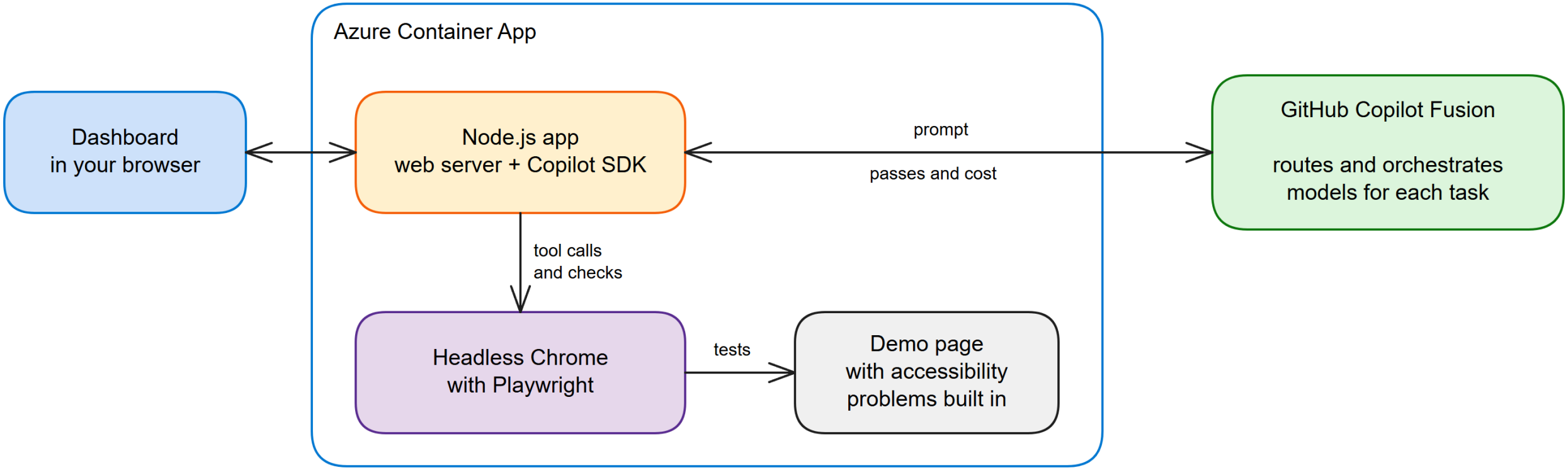
Chat shows the workflow it picked, then each pass as it runs.

Pattern	Chat says	Passes
Single	Selected Single workflow	Main
Critique	Selected Critique workflow	First, Critique, Revision
Cascade	Selected Cascade workflow	Main, Review, Fix-up

Revision and Fix-up only show up when the review asks for changes.

Try the same prompt at High and Max effort, in new chats, and compare.

Demo Architecture



Demo



https://aka.ms/hydrafusion

Demo



https://github.com/roryp/hydrafusion